Reproduction of OpenWAM-α on LIBERO

Ego Research

October 10, 2026

Abstract

This report documents a closed-loop reproduction of the released OpenWAM-α LIBERO policy on a single A100 GPU. Using the authors' native task protocol and published weights, the policy reaches $1985/2000 = 99.25\,\%$ over the four LIBERO suites, 0.05 percentage points below the officially reported $99.30\,\%$. No weights were retrained. The report covers the benchmark, the model architecture, the data pipeline, the frozen environment, the aggregate and per-task results, all 15 failed episodes, compute and timing figures, a genuine teaching case, and the limitations of the claim.

Contents

1 Experiment A: OpenWAM-α on LIBERO **2**
- 1.1 Overview and Scope . . . 2
- 1.2 The LIBERO Benchmark . . . 2
- 1.3 Model and Inference Architecture . . . 3
- 1.4 Closed-Loop Data Pipeline . . . 3
- 1.5 Frozen Environment and Provenance . . . 4
- 1.6 Results . . . 5
- 1.7 Compute, Timing and Control Budget . . . 8
- 1.8 Failure Inventory . . . 9
- 1.9 Reproducibility Notes . . . 9
- 1.10 Teaching Case: Images, Actions and Video . . . 9
- 1.11 Discussion: Reading OpenWAM Alongside $\pi_{0.5}$ . . . 12
- 1.12 Limitations . . . 13
- 1.13 Summary . . . 13

1 Experiment A: OpenWAM-α on LIBERO

1.1 Overview and Scope

This experiment reproduces the published LIBERO success rate of the *OpenWAM-α* policy [2] on a single A100 GPU. No new weights were trained and no per-seed tuning was performed: the authors' released LIBERO checkpoint is evaluated under its native task protocol, and the resulting score is compared against the officially reported number.

LIBERO contains 130 language-conditioned manipulation tasks in total. This evaluation covers the four standard suites — Spatial, Object, Goal and LIBERO-10 — that together contain 40 tasks. Each task is rolled out for 50 episodes, giving $40 \times 50 = 2000$ closed-loop episodes. The complete task definitions, scene assets and fixed initial states are taken from a pinned version of LIBERO. Expert demonstrations are not required for evaluation and the authors' full training set was not downloaded.

1.2 The LIBERO Benchmark

LIBERO is a language-conditioned manipulation benchmark for simulated robot arms [1]. Every task defines a tabletop scene, a natural-language goal and a BDDL goal predicate. At each control step the policy observes the current external and wrist RGB frames together with the robot's proprioceptive state, predicts an action, and the MuJoCo simulator advances the scene to produce the next observation. This is a closed loop, not video classification and not a real-robot deployment.

The raw data therefore consists of scene assets, BDDL task descriptions, language instructions and fixed initial states. RGB frames are rendered dynamically from the geometry, joint, mass, friction, contact and camera configuration of the simulator; the model is never handed a pre-recorded future video. Object ground-truth positions and the success predicate are maintained by the environment and are not extra policy inputs.

A few terms are used with a precise meaning throughout this report:

- **task** — the goal, for example *"put the milk in the basket"*;
- **trial / episode** — one complete attempt starting from a fixed initial state;
- **control step** — one execution of a 7-dimensional arm command;
- **query** — one network planning call, which may emit several actions at once.

Trial IDs 0–49 index the 50 attempts of a task and select the initial state; they are *not* seeds. A seed is the origin of a random-number stream; fixing it aids reproducibility but is not by itself sufficient — initial states, library versions, numerical precision and model stochasticity must be fixed as well.

The final LIBERO action is a 3-dimensional translation command, a 3-dimensional rotation command and a 1-dimensional gripper command. It is not a set of seven joint angles, and the numbers cannot be read directly as metres. At a 20 Hz control rate each step corresponds to roughly 0.05 s of simulated time; internal physics sub-steps are not counted as policy steps. A budget of 600 or 700 steps therefore corresponds to about 30 or 35 seconds of simulated time, which is unrelated to wall-clock runtime. The budget is the maximum number of control steps allowed — not a monetary or GPU-capacity limit. An episode stops as soon as the goal is satisfied; if the budget is exhausted the episode is recorded as a failure. The initial stabilisation wait is not charged against the policy budget.

Success is measured by the task-specific scoring rule, which inspects spatial relations, containment relations and joint or switch states of the objects. The policy sees the natural-language goal while the scorer reads the machine-checkable goal predicate. Success rate is defined as successful episodes divided by all episodes; with 50 trials per task and 500 per suite, and with equal denominators across the four suites, the mean of the suite scores equals the overall 2000-episode success rate. All failures are retained and engineering errors are reported separately; incomplete runs are never presented as full results.

Official summary scores are not a per-episode ground-truth archive. Agreement with the official number does not imply sample-by-sample identity, and because the two models reproduced in this report

use different native budgets, seeds, simulator versions and replanning rates, the difference between their scores cannot be interpreted as a fair architectural advantage.

1.3 Model and Inference Architecture

The checkpoint used here is `OpenWAM-Alpha-Sim-LIBERO/checkpoint_step_10690.safetensors`, pinned to Hugging Face revision `110b07e4d5581c81520a77419cbb0e9371df231b`. All nine files (24,835,230,170 bytes in total) were verified; the main weight file is 24,813,767,464 bytes with SHA-256 `34c9a6c9af67dda9aaffa2edc0efa29b28eeee1c26dea419f98065c80508c96a`.

The loading log confirms a Wan video DiT backbone, the WanVideoVAE38 decoder, a Wan text encoder and a dedicated action expert, composed through the authors' *Dual-System Joint Self-Attention* with `bfloat16` mixed precision. A Wan2.2-TI2V-5B video base [3] and the action branch are combined by joint self-attention, so that a video-generation backbone participates in world/action modelling — a design that differs from the vision-language backbone plus action expert used by $\pi_{0.5}$ (Experiment B). The dual-system joint modelling and the unified action representation are the authors' contributions, not contributions of this report.

Internally the model works in an 80-dimensional unified action representation. A normaliser maps this representation to and from a native 10-dimensional end-effector action, and the client converts the rotation part into LIBERO's 7-dimensional control, so the first seven entries cannot simply be sliced off.

Figure 1 lists the runtime data flow, which is a deployment diagram rather than a layer-by-layer network topology.

task language → author tokenizer / text encoder;
external + wrist RGB → two-view canvas / video encoder;
EEF 10-D state → min-max normalisation / 80-D mapping;
(text, video, state) → joint denoising of video backbone and action expert;
→ native EEF 10-D action → rotation conversion → 7-D LIBERO control;
→ execute and re-observe (feedback loop).

Figure 1 Runtime data flow of the OpenWAM-α closed loop.

The server logs report a 384×320 canvas with multi-view enabled; two cameras are actually present and the third slot is empty. Denoising uses a 10-step schedule in synchronous mode with a deployment horizon of 10. The authors additionally enable a DiT velocity cache with a threshold of 0.990, so the ten denoising steps must not be described as ten independent full video-DiT forward passes.

What is evaluated here is action success rate, not future-video quality. The full run stored no predicted future RGB and no future ground truth, so no world-model video metric, counterfactual accuracy or causal effect is reported. The fact that the architecture is a world–action model does not by itself demonstrate such capabilities in this experiment.

1.4 Closed-Loop Data Pipeline

A single episode proceeds as follows.

1. Task definitions, language, scene assets and initial states are read from the pinned LIBERO repository. Judging the released weights does not require the full training demonstration set, and this run did not download it.

2. One simulator instance is created per task and trials 0–49 are executed consecutively; after each reset the corresponding initial state is applied. The environment seed is 42.

3. Thirty stabilisation steps are executed and are not charged against the policy budget (600 for Spatial/Object/Goal, 700 for LIBERO-10).

4. External and wrist RGB images and the robot state are captured. The author-side end-effector state is a 3-D position, a continuous 6-D rotation and a 1-D gripper value. Object ground-truth fields exist in the simulator observation but were not fed to the policy.

5. The client packages images, task and state and sends the PNG payload over a WebSocket to a separate model process. The teaching hook records the genuine PNG bytes; no camera layout or $\pi_{0.5}$-style state adaptation is substituted.

6. The server applies min-max normalisation from the checkpoint's `normalization_stats.npy`, runs the joint denoising, and hands the output to the adapter.

7. The client converts the 6-D rotation of the native 10-D action into LIBERO rotation control to form a 7-D action and executes it at the author's caching cadence; typically a 10-step chunk is consumed before replanning. The 74-control-step teaching case corresponds to 8 planning calls — control steps are not neural inference calls.

8. Each action changes the scene and the next observation re-enters the loop. `env.done` is the success signal; episodes that time out are still counted in the denominator.

9. Per-task `results.json` files and the client `RESULT` log store every trial, its success flag, policy steps and terminal reward; the run is then summarised into `summary.json` and `summary.csv`.

Debug logging was disabled for the full run. Ninety log, configuration and result files were retained, but per-query PNGs, action chunks and states were not stored for the main table. A single teaching case additionally stores 74 genuine observation/action frames; it is a learning exhibit and does not replace the 2000-episode result.

1.5 Frozen Environment and Provenance

Table 1 Frozen configuration of the full OpenWAM-α run.

Item	Value
Author source	`48bd67b89d489b14d03b8d92bc66e65d306df32e`
LIBERO source	`8f1084e3132a39270c3a13ebe37270a43ece2a01`
Python (model/client)	3.11.16
PyTorch (model)	2.5.1+cu121
Transformers	5.5.0
NumPy	1.26.4
MuJoCo / robosuite	3.3.2 / 1.4.0
dtype / attention	`bfloat16` / Torch SDPA
`compile`	false
TF32	off, precision *highest*, `NVIDIA_TF32_OVERRIDE=0`
Hardware	single A100, EGL, model and simulator in separate processes
Model server replicas	1, port 8920, concurrency unchanged mid-run
Environment seed / trial	42 / 0–49 per task
Checkpoint saving	none (inference only, no new weights)

The authors recommend Python 3.10 and Torch 2.7; this run reuses an existing Python 3.11 / Torch 2.5.1 environment with SDPA and `compile` disabled. These deviations are disclosed and no claim of bitwise equality, equal speed or a fully reproduced author environment is made. System drivers, CUDA, the base Conda environment and the global shell were left untouched.

The formal run ID is `openwam-w2-libero-native-full-s42`. The pre-registered source Git revision was `0c6e95571585a29914f62e7036768be57379482d` and the queue file SHA-256 was `1078f11d00808fe962c6215e03f577499e219136dcb0d9cf6b1140bb495a25f`. The executed launch command was:

```
server-python OpenWAM/benchmarks/libero/run_all_suites.py \
  --ckpt-dir <models>/OpenWAM-Alpha-Sim-LIBERO \
  --ckpt-name checkpoint_step_10690.safetensors \
  --server-python <server-python> --libero-python <client-python> \
  --libero-path <LIBERO-8f1084e3...> \
  --policy-config <OpenWAM>/benchmarks/libero/policy_config.yml \
  --gpus 0 --render-gpus 0 --replicas-per-gpu 1 \
  --compile-enabled false --client-max-attempts 1 \
  --output-dir <runs>/openwam-w2-libero-native-full-s42/output
```

Re-evaluating the policy requires a new unique run ID, a fresh space/ETA registration and a new configuration; this output must not be overwritten.

1.6 Results

The full 2000-episode run completed with 1985 successes and 15 failures, giving an overall success rate of 99.25 %. Table 2 breaks the result down by suite and compares it with the authors' reported numbers.

Table 2 Per-suite success rate: this reproduction versus the official OpenWAM-α scores.

Suite	Success / total	This run	Official	Difference (pp)
libero_spatial	497 / 500	99.40 %	99.60 %	−0.20
libero_object	498 / 500	99.60 %	99.60 %	+0.00
libero_goal	497 / 500	99.40 %	99.80 %	−0.40
libero_10	493 / 500	98.60 %	98.20 %	+0.40
Mean / total	**1985 / 2000**	**99.25 %**	**99.30 %**	**−0.05**

The official table publishes only summary scores and provides no per-episode predictions, so sample-by-sample agreement with the authors cannot be claimed. The value 99.25 % is reported to two decimals and is not described as exactly matching 99.30 % merely because of rounding. A single fixed seed cannot demonstrate multi-seed stability, statistical superiority or a new method. Figure 2 shows the aggregate comparison and Figure 3 the per-task breakdown.

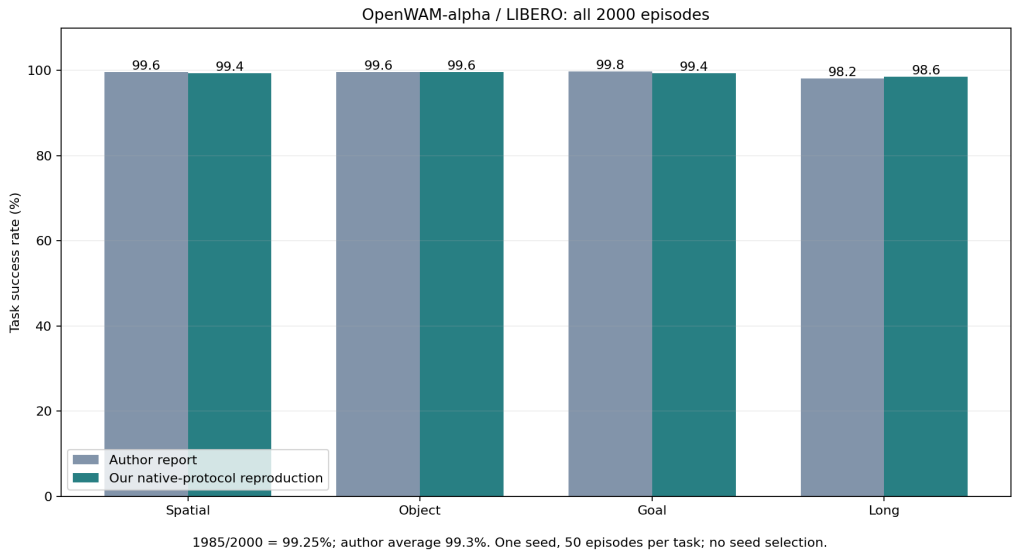


Figure 2 Official OpenWAM-α scores versus this reproduction.

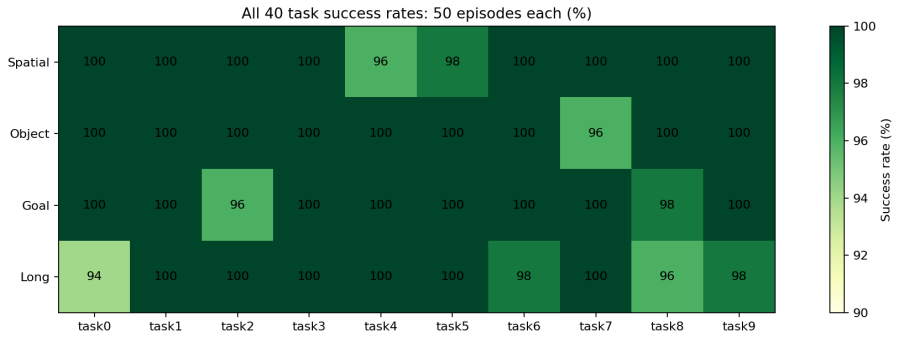


Figure 3 Per-task success rate over all 40 LIBERO tasks.

Table 3 lists every task together with the number of successful episodes and the failing trial IDs.

Table 3 Per-task results for the full OpenWAM-α run.

Suite	ID	Task language	Succ./50	Failing trials
libero_spatial	0	pick up the black bowl between the plate and the ramekin and place it on the plate	50	—
libero_spatial	1	pick up the black bowl next to the ramekin and place it on the plate	50	—
libero_spatial	2	pick up the black bowl from table center and place it on the plate	50	—
libero_spatial	3	pick up the black bowl on the cookie box and place it on the plate	50	—
libero_spatial	4	pick up the black bowl in the top drawer of the wooden cabinet and place it on the plate	48	4, 27
libero_spatial	5	pick up the black bowl on the ramekin and place it on the plate	49	2
libero_spatial	6	pick up the black bowl next to the cookie box and place it on the plate	50	—
libero_spatial	7	pick up the black bowl on the stove and place it on the plate	50	—
libero_spatial	8	pick up the black bowl next to the plate and place it on the plate	50	—
libero_spatial	9	pick up the black bowl on the wooden cabinet and place it on the plate	50	—
libero_object	0	pick up the alphabet soup and place it in the basket	50	—
libero_object	1	pick up the cream cheese and place it in the basket	50	—
libero_object	2	pick up the salad dressing and place it in the basket	50	—
libero_object	3	pick up the bbq sauce and place it in the basket	50	—
libero_object	4	pick up the ketchup and place it in the basket	50	—
libero_object	5	pick up the tomato sauce and place it in the basket	50	—
libero_object	6	pick up the butter and place it in the basket	50	—
libero_object	7	pick up the milk and place it in the basket	48	10, 13
libero_object	8	pick up the chocolate pudding and place it in the basket	50	—
libero_object	9	pick up the orange juice and place it in the basket	50	—
libero_goal	0	open the middle drawer of the cabinet	50	—
libero_goal	1	put the bowl on the stove	50	—
libero_goal	2	put the wine bottle on top of the cabinet	48	38, 47
libero_goal	3	open the top drawer and put the bowl inside	50	—
libero_goal	4	put the bowl on top of the cabinet	50	—
libero_goal	5	push the plate to the front of the stove	50	—
libero_goal	6	put the cream cheese in the bowl	50	—
libero_goal	7	turn on the stove	50	—
libero_goal	8	put the bowl on the plate	49	25
libero_goal	9	put the wine bottle on the rack	50	—
libero_10	0	put both the alphabet soup and the tomato sauce in the basket	47	7, 14, 38
libero_10	1	put both the cream cheese box and the butter in the basket	50	—
libero_10	2	turn on the stove and put the moka pot on it	50	—
libero_10	3	put the black bowl in the bottom drawer of the cabinet and close it	50	—
libero_10	4	put the white mug on the left plate and put the yellow and white mug on the right plate	50	—
libero_10	5	pick up the book and place it in the back compartment of the caddy	50	—
libero_10	6	put the white mug on the plate and put the chocolate pudding to the right of the plate	49	23
libero_10	7	put both the alphabet soup and the cream cheese box in the basket	50	—
libero_10	8	put both moka pots on the stove	48	29, 38
libero_10	9	put the yellow and white mug in the microwave and close it	49	25

Independent verification read all 40 `results.json` files and matched each task against the 50 client `RESULT` entries on trial ID, success flag and policy steps, additionally checking seed, stabilisation steps, budget, end-effector mode, coverage, reward, summary and manifest. All 2000 unique trials, 1985 successes and 15 failures agree. The 90 formal files (10,271,072 bytes) were returned with per-file size/SHA checks and a server acknowledgement fingerprint of `ca759e55b79bb6c7556399a685662773481eb07f91d908fd62c6d11d4cf91f21`. Every complete episode retains only its task, trial, outcome, action count and terminal reward — not the full action sequence, frame-by-frame media or environment state.

1.7 Compute, Timing and Control Budget

The formal run started at 2026-10-05T18:40:58+08:00 and finished at 2026-10-06T01:51:27+08:00 with exit code 0. The total wall time was 25,829.198 seconds (7 h 10 m 29 s), including model loading, compilation and caching, simulation, serialisation, task switching and log writing, but excluding model download, earlier failed attempts and document preparation.

Full-device sampling at 2-second intervals yielded 12,617 points, with a peak memory of 25,676 MiB ($\approx$ 25.07 GiB) and an average GPU utilisation of 29.40 %. A momentary high utilisation does not mean the GPU was saturated for the whole run, and the sampled peak may miss short spikes between samples.



Figure 4 Measured GPU memory and utilisation over the full run.

A single model replica used about 25 GiB, which suggests that two replicas would fit on the device, but whether that would reduce wall time still requires a controlled measurement under a fixed protocol; a linear speed-up is not promised. Any future increase in concurrency must record the model's random streams and task assignment and must not silently change the replica count of this frozen run.

Table 4 Executed control steps per suite.

Suite	Total control steps	Median per episode
libero_spatial	51,855	100.5
libero_object	66,294	129.0
libero_goal	56,520	92.0
libero_10	127,141	238.0
Total	**301,810**	—

The 301,810 executed control steps are not equal to the number of model queries; the full run has no per-query audit, so no exact call count or pure-GPU throughput is invented.

1.8 Failure Inventory

All 15 failures are listed in Table 5. Each exhausted its budget (600 steps for Spatial/Object/Goal, 700 for LIBERO-10) and ended with a terminal reward of 0.0.

Table 5 All 15 failed episodes of the full run.

Suite	Task	Trial	Control steps	Terminal reward
libero_spatial	4	4	600	0.0
libero_spatial	4	27	600	0.0
libero_spatial	5	2	600	0.0
libero_object	7	10	600	0.0
libero_object	7	13	600	0.0
libero_goal	2	38	600	0.0
libero_goal	2	47	600	0.0
libero_goal	8	25	600	0.0
libero_10	0	7	700	0.0
libero_10	0	14	700	0.0
libero_10	0	38	700	0.0
libero_10	6	23	700	0.0
libero_10	8	29	700	0.0
libero_10	8	38	700	0.0
libero_10	9	25	700	0.0

The failures span drawer retrieval, dairy placement, placing a bottle on a cabinet, putting a pot on the stove and combined operations, but a task name is not a diagnosed failure mechanism. Without the full video and action record of each failing episode, the failures cannot be attributed to occlusion, memory, grasping, planning or model hallucination. Any future diagnosis must register a new run and new random conditions; new footage from a different random stream cannot be presented as these 15 original failure trajectories.

1.9 Reproducibility Notes

The first smoke attempt failed because the author implementation rejected the `torch_sdpa` attention enum, producing 0 episodes; after switching to the legal `torch` value, a second attempt still produced 0 episodes because `pandas` was missing during normaliser import. Both failures and their logs were kept.

The root cause is that three packages eagerly re-export a training-only LeRobot reader alongside the inference normaliser. Because the private disk lacked room to install the full `pandas`/`pyarrow` training stack, only the three initialisers were changed to lazy loading in a separate copy of the author source; the remaining 546 files (including the model and the normalisation maths) are SHA-identical, and a real CPU normaliser import and min–max round trip passed. The tool is provided as `prepare_inference_imports.py`. On 2026-10-06 the same 546 files and three initialisers were re-verified.

The teaching retry then succeeded (1/1, 74 steps) and the full run completed once with exit code 0 and `client_max_attempts=1`; no best-of-retry results were spliced into the main table. TCP readiness probes during server start-up produced `InvalidMessage` handshake log lines, which are not failed policy tasks — log lines containing the word "error" are not automatically treated as NaN.

1.10 Teaching Case: Images, Actions and Video

The teaching case fixes Spatial task 0, initial state 0 and succeeded once in 74 control steps with 8 planning calls. All native 10-D end-effector, 7-D LIBERO and 10-D state values are finite. It is not an extra 2001st episode of the main table. The input images are the data the model genuinely received, not paper screenshots or generated images.

The state is a 3-D position, a 6-D rotation and a 1-D gripper value ($3 + 6 + 1 = 10$); the internal 80-D representation is a unified model format, not 80 robot degrees of freedom. The following record is

the genuine first control step of the teaching case. Note that the native action and the final control differ in rotation and gripper due to adaptation, and that the final 7-D action is not the first seven entries of the native 10-D action.


```
{
  "state": [
    -0.21130245923995972, -0.010944196954369545, 1.1741831302642822,
    0.9984198808670044, -0.00016182575200218707, -0.056193094700574875,
    -0.0001631764171179384, -1.0, -1.944792347785551e-05, 0.975337028503418
  ],
  "native_eef10_action": [
    0.3204345405101776, 0.3149413764476776, 0.1583862155675888,
    0.99609375, 0.0078125, -0.0302734375, -0.0146484375,
    0.99609375, -0.0087890625, 1.0078125
  ],
  "libero_action7": [
    0.3204345405101776, 0.3149413764476776, 0.1583862155675888,
    -0.009145314805209637, 0.030346114188432693, 0.007979447022080421, -1.0
  ]
}
```


Figure 5 shows the two genuine input views of the case, and Figure 6 plots the executed actions together with the end-effector position.

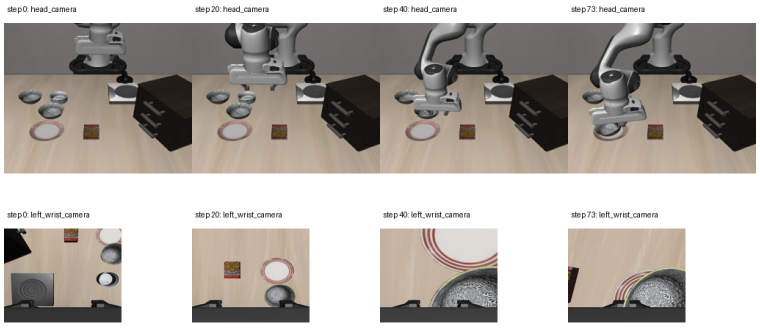


Figure 5 Two genuine input views of the teaching case.

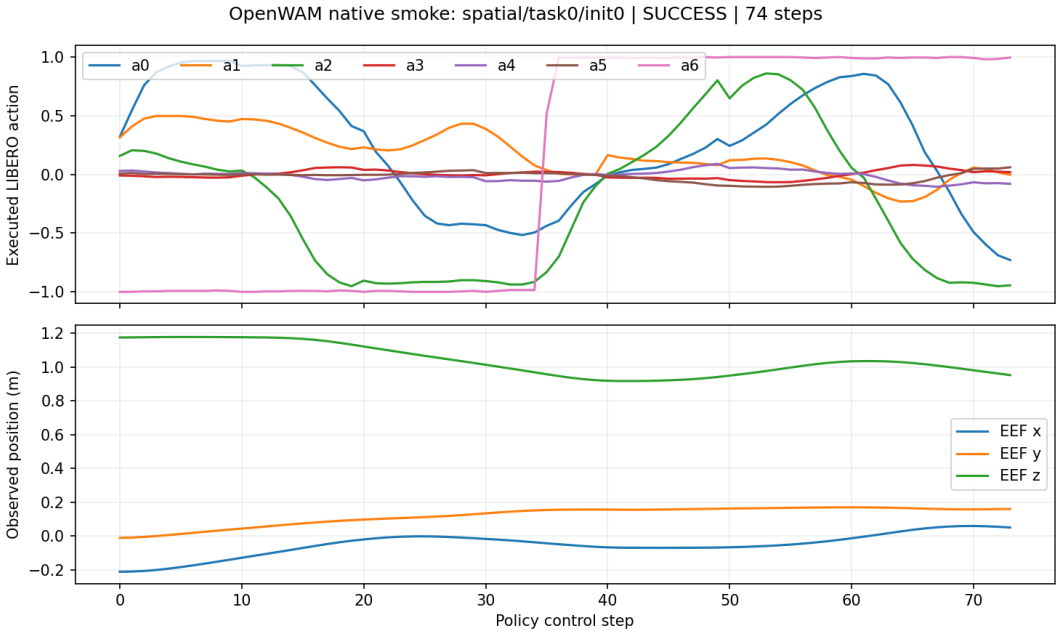
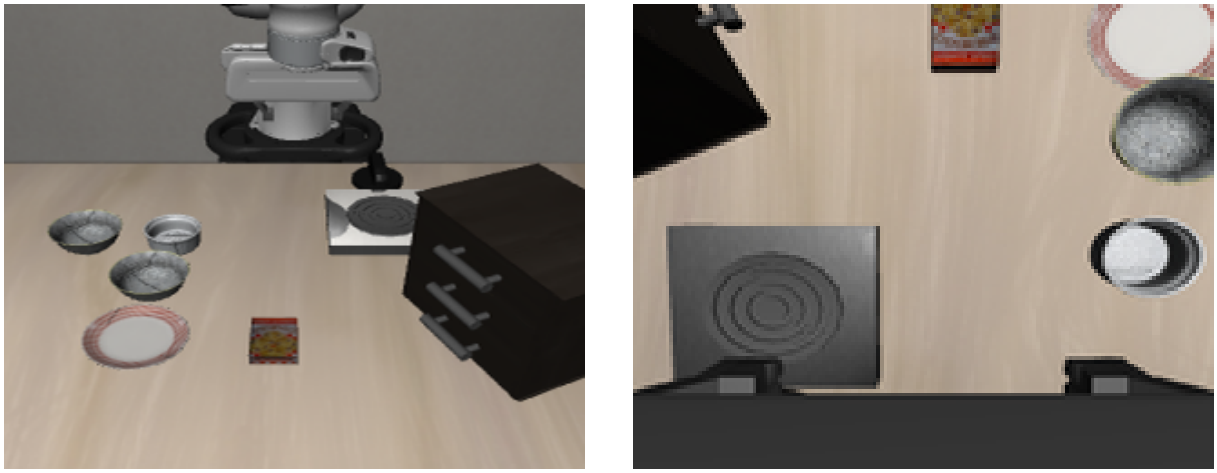


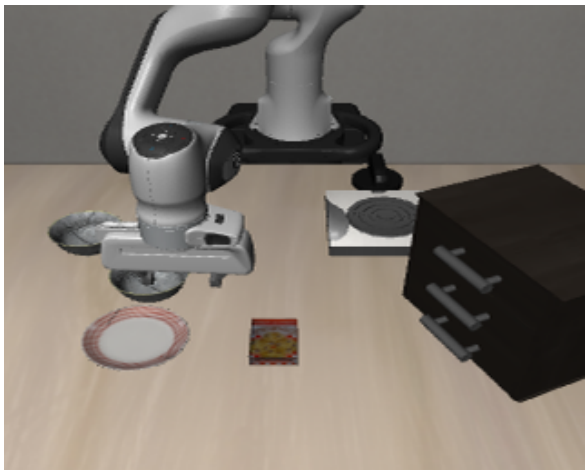
Figure 6 Executed actions and end-effector position of the teaching case.

Figures 7, 8 and 9 show the two camera views at three points of the episode: before the first action, at step 36 and at step 73. The saved frames precede their actions, and success is decided by the post-action environment signal rather than by the last image.

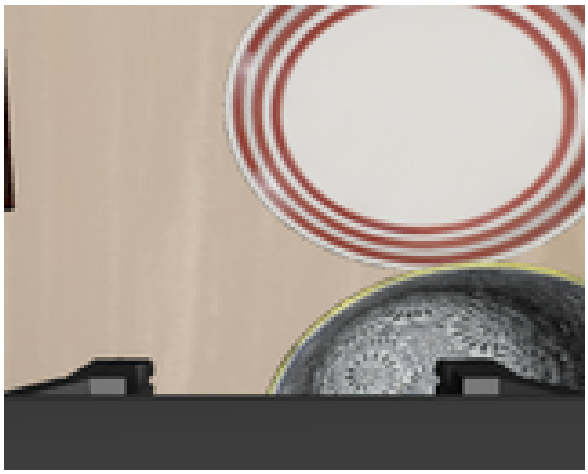


(a) external view **(b)** wrist view

Figure 7 Observation before the first action (step 0).

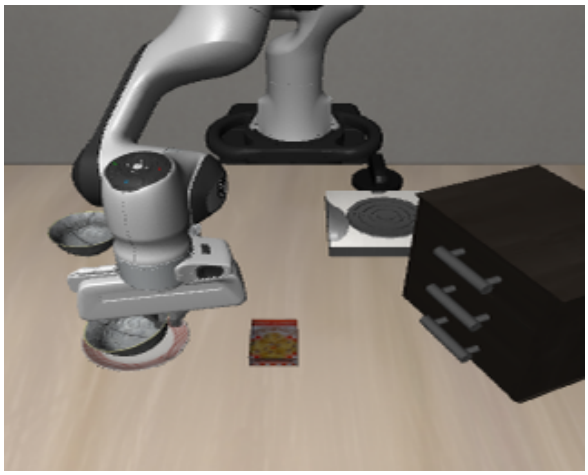


(a) external view

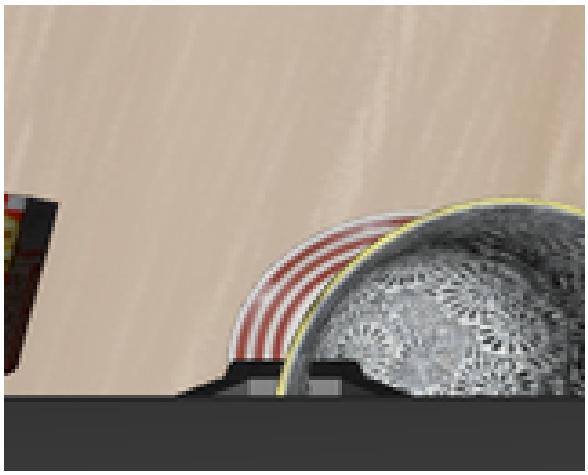


(b) wrist view

Figure 8 Observation before the action at step 36.



(a) external view



(b) wrist view

Figure 9 Observation before the last action (step 73).

The corresponding 74-step two-camera video is stored alongside the figures as `case-rollout.mp4` and can be opened in a local player. It is the genuine smoke case from before the full run, not an author demo or a generated video, and not an original recording of the W2 episodes. The `head_camera` field is the simulator's external camera, not a human-worn camera.

1.11 Discussion: Reading OpenWAM Alongside $\pi_{0.5}$

Table 6 places the two reproductions side by side.

Table 6 OpenWAM-α versus $\pi_{0.5}$, each under its own native protocol. The two columns answer separate reproduction questions.

Item	$\pi_{0.5}$ (Exp. B)	OpenWAM-α (Exp. A)
Mean success rate	96.75 %	99.25 %
Official mean	96.85 %	99.30 %
Offset from official	−0.10 pp	−0.05 pp
Environment seed	7	42
Stabilisation wait	10	30
Spatial/Object/Goal budget	220/280/300	600/600/600
LIBERO-10 budget	520	700
Execution per plan	predict 10, execute first 5	native cache cadence, horizon 10
MuJoCo	3.2.3	3.3.2
Per-query audit	saved and independently verified	debug off, no equivalent record

The two evaluations share the same four LIBERO suites, but the models, initial random conditions, time budgets, control cadence and simulator versions were not unified. The two columns therefore answer each author's own reproduction question, and the difference $99.25\,\% - 96.75\,\% = 2.50$ percentage points must not be presented as a fair architectural gain. A model-to-model comparison would require a separate pre-registered protocol that fixes the environment, seed, initial states, budget, action cadence and cost. The high score obtained here is a baseline reproduction, not a new method.

1.12 Limitations

The claims of this experiment are bounded as follows. Evaluation is closed-loop action success in simulation only; no real-robot deployment is attempted. World-model video quality is not measured. Only one seed and one initial-state set per task are used, so multi-seed stability is not established. The full run keeps no per-query media, which limits post-hoc failure analysis. The environment differs from the authors' recommendation in Python and Torch versions. The official numbers are summary scores with no per-episode archive, so exact agreement with the authors cannot be verified.

1.13 Summary

On a single A100 with the authors' released weights and native protocol, OpenWAM-α reproduces $1985/2000 = 99.25\,\%$ on the four LIBERO suites, 0.05 percentage points below the official $99.30\,\%$. All 15 failures are retained, the environment and provenance are documented, and a genuine teaching case is provided with images, actions and a video. The result is a usable, verifiable baseline of the authors' policy together with engineering and reproducibility experience — not a new contribution, and not automatically transferable to human egocentric video.

References

[1] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.

[2] OpenWAM Authors. An open, modular exploration towards systematic world–action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026.

[3] Wan Team. Wan: Open and advanced large-scale video generative models. *arXiv preprint*, 2025.